Page Object Model and Reusable Framework Design

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–11 concepts (Locators, web-first assertions, auto-waiting, fixtures,
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Architect maintainable, enterprise-grade Playwright frameworks using the Page Object Model (POM) pattern in pure modern JavaScript. Decouple page interaction mechanics from business test logic.

Day 12 Learning Objectives

'Understand why framework architecture matters: separation of concerns, maintainability, and readability.
'Implement the Page Object Model (POM) using modern JavaScript ES6 classes.
'Encapsulate page locators as class properties and interaction workflows as async methods.
'Keep business assertions inside test files to preserve clear reporting and failure traceability.
'Organize production Playwright projects: pages/, tests/, fixtures/, test-data/, and utils/.
'Avoid common POM anti-patterns: oversized "God Objects", unnecessary inheritance, and duplicate helpers.
'Inject Page Objects cleanly into tests using Playwright custom fixtures (test.extend).

In an enterprise software team, automated tests are production code. Without clear architectural patterns, test suites quickly devolve into fragile, copy-pasted scripts that nobody wants to touch.

The Problem with Script-Based Automation
If 30 test files directly interact with low-level page locators:
```javascript
// In test 1:
await page.getByRole('button', { name: 'Log In' }).click();
// In test 2:
await page.getByRole('button', { name: 'Log In' }).click();
// In test 30:
await page.getByRole('button', { name: 'Log In' }).click();
```
If the button label changes to "Sign In", you must manually edit 30 test files!

Separation of Concerns
1. **The Test File (.spec.js): Focuses on **business intent, user journey flow, and assertions**. It asks: "Did the order complete? Was the discount applied?"**
2. **The Page Object (.js): Focuses on **page interaction mechanics**. It knows: "Which locator identifies the checkout button? How do I fill the shipping address form?"**

The Maintainability Trade-Off
Abstractions come with a cost: creating too many small helper layers adds mental overhead. Apply Page Objects where pages have significant reuse across multiple tests (login, navigation, cart, checkout).
Comparing Raw Scripts with Page Objects (Showing how a test file reads like plain English when backed by a Page Object.)

```
// Raw script (cluttered with DOM details)
await page.getByPlaceholder('Email').fill('user@apex.io');
await page.getByPlaceholder('Password').fill('Pass123!');
await page.getByRole('button', { name: 'Sign In' }).click();

// With Page Object Model (clean, readable business intent)
const loginPage = new LoginPage(page);
await loginPage.login('user@apex.io', 'Pass123!');
```

In pure modern JavaScript, a Page Object is an ES6 class that accepts the Playwright `page` object in its constructor.

Anatomy of a Page Object Class
```javascript
// pages/LoginPage.js
class LoginPage {
  /**
   * @param {import('@playwright/test').Page} page
   */
```

```
  constructor(page) {
    this.page = page;

    // Locators defined as class properties
    this.emailInput = page.getByLabel(/email/i);
    this.passwordInput = page.getByLabel(/password/i);
    this.submitButton = page.getByRole('button', { name: /sign in|log in/i });
    this.errorMessage = page.getByRole('alert');
  }
  async goto() {
    await this.page.goto('/login');
  }
  async login(email, password) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}

module.exports = { LoginPage };
```

Rule: Keep Assertions in the Test File
A controversial debate in test automation is whether assertions belong inside Page Objects.
In modern Playwright engineering, **assertions belong in the test file (`.spec.js`), NOT in the Page Object!

Why?

1. **Reporting Clarity:** When an assertion fails, the test runner points directly to the line in the test spec, making it immediately clear which business requirement failed.
2. **Reusability:** A method like `login()` can be used in both positive tests (expecting success) and negative tests (expecting failure). If `login()` asserts success internally, you cannot reuse it for negative tests!

Using the Page Object in a Spec File (A clean spec file consuming LoginPage.)

```
const { test, expect } = require('@playwright/test');
const { LoginPage } = require('../pages/LoginPage');

test('successful student login', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('student@rootaccess.org', 'ValidPass123!');

  // Assertion stays in the test spec!
  await expect(page).toHaveURL(/.*dashboard/);
});
```

A well-structured Playwright project scales gracefully from 5 tests to 500 tests.

Recommended Directory Layout

```text
rootaccess-automation/
% % %  playwright.config.js      # Central test runner configuration
% % %  package.json
% % %  pages/                   # Page Object classes
%    % % %  BasePage.js          # Optional common navigation/header methods
%    % % %  LoginPage.js
%    % % %  CatalogPage.js
%    % % %  CheckoutPage.js
%    % % %  DashboardPage.js
% % %  fixtures/                # Custom fixtures injecting Page Objects
%    % % %  testFixtures.js      # test.extend() wiring page objects
% % %  test-data/               # Test data sets & personas
%    % % %  users.json
%    % % %  products.json
% % %  utils/                   # Generic utilities (formatting, date helpers)
%    % % %  dateUtils.js
% % %  tests/                   # Test specifications organized by feature
   % % %  auth/
```

% % % % login.spec.js
% % % % registration.spec.js
% % % catalog/
% % % % course_search.spec.js
% % % checkout/
% % % purchase_flow.spec.js
```

**Clean Page Object Injection with `test.extend`**
**Instead of writing `const loginPage = new LoginPage(page)` in every single test, create a custom fixture:**
**```javascript**
**// fixtures/testFixtures.js**
**const base = require('@playwright/test');**
**const { LoginPage } = require('../pages/LoginPage');**
**const { CatalogPage } = require('../pages/CatalogPage');**
const test = base.test.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  catalogPage: async ({ page }, use) => {
    await use(new CatalogPage(page));
  },
});

module.exports = { test, expect: base.expect };
```

Now in tests, simply destructure `{ loginPage, catalogPage }` directly from the test parameters!

Writing Tests with Injected Page Object Fixtures (A test utilizing custom fixtures.)

```
const { test, expect } = require('../fixtures/testFixtures');

test('search course catalog', async ({ catalogPage }) => {
  await catalogPage.goto();
  await catalogPage.search('Playwright');
  await expect(catalogPage.firstResultTitle).toContainText('Playwright Automation');
});
```

Even when using Page Objects, poor design decisions can undermine framework maintainability.

Anti-Pattern 1: The "God Object"
Creating a single `App.js` or `MasterPage.js` with 1,500 lines containing every locator in the application.
****Fix:** Keep Page Objects focused strictly on a single page or logical component (e.g. `Navbar.js`, `CartDrawer.js`, `CheckoutPage.js`).**

Anti-Pattern 2: Deep Inheritance Hierarchies
Creating `BasePage` extends `CorePage` extends `AbstractComponent` extends `Root`.
In modern JavaScript, deep inheritance creates brittle coupling where changes to base classes break unrelated pages.
****Fix:** Prefer **Composition over Inheritance**. For shared UI components (like a Header or Footer), create a separate `HeaderComponent.js` class and instantiate it inside relevant Page Objects.**

Anti-Pattern 3: Wrapper Methods for Basic Actions
Writing methods like `clickButton(locator)` or `typeText(locator, text)` that simply wrap Playwright's native methods.
Playwright's API is already high-level, auto-waiting, and expressive. Don't wrap `await page.locator.click()` in a custom function unless it adds meaningful domain logic!

Component Composition Example (Embedding a reusable Header component inside multiple page objects.)

```
// pages/components/HeaderComponent.js
class HeaderComponent {
  constructor(page) {
    this.page = page;
    this.userMenu = page.getByRole('button', { name: /user menu/i });
    this.cartBadge = page.getByTestId('cart-count');
  }
}

// pages/HomePage.js
class HomePage {
  constructor(page) {
    this.page = page;
    this.header = new HeaderComponent(page);
    this.heroBanner = page.getByRole('banner');
  }
}
```

requirements.

Assignment Starter Code (Day 12)

```
// tests/shopping_pom_refactored.spec.js
const { test, expect } = require('@playwright/test');

// ============================================================================
// 1. PAGE OBJECT: LoginPage
// ============================================================================
class LoginPage {
  /**
   * @param {import('@playwright/test').Page} page
   */
  constructor(page) {
    this.page = page;
    this.emailInput = page.getByLabel(/email/i);
    this.passwordInput = page.getByLabel(/password/i);
    this.signInButton = page.getByRole('button', { name: /sign in|log in/i });
    this.errorMessage = page.getByRole('alert');
  }

  async goto() {
    await this.page.goto('/login');
  }
```

DAY 12 INTERVIEW PREPARATION QUESTION BANK

Q1. What is the Page Object Model (POM), and why is it considered an industry standard in test automation?

[Difficulty: Beginner] Evaluation Focus: Design pattern fundamentals and maintainability principles.

Model Answer: The Page Object Model (POM) is an architectural design pattern where web pages or components are represented by classes. The class properties store element locators, and class methods encapsulate user interactions (e.g. login, search, checkout). Tests interact with the page through this abstraction rather than directly manipulating HTML selectors. If the UI changes, we update the selector once in the Page Object class, and all referencing tests continue to pass without modification.

Follow-Up Question: "How does POM improve test readability for non-technical stakeholders?"

Q2. Should web-first assertions (expect) live inside Page Object methods or inside test specification files? Why?

[Difficulty: Intermediate] Evaluation Focus: Separation of concerns and test reporting clarity.

Model Answer: In modern Playwright frameworks, assertions should reside in the test specification files (`.spec.js`), not inside Page Objects. Page Objects should represent the state and behavior of the application, while test specs verify that the behavior matches business expectations. Keeping assertions in test specs enables clear failure reporting (the failure stack trace points to the exact test case requirement) and allows interaction methods—such as `login(user, pass)`—to be reused across both positive success tests and negative validation tests.

Follow-Up Question: "Is it ever acceptable to have an assertion inside a Page Object?"

Q3. How do you inject Page Objects into Playwright tests using custom fixtures instead of manual class instantiation?

[Difficulty: Intermediate] Evaluation Focus: Playwright test.extend architecture and dependency injection.

Model Answer: We use Playwright's `test.extend()` API to create custom fixtures. In a `fixtures/testFixtures.js` file, we extend the base `test` by defining fixtures for each Page Object (e.g. `loginPage: async ({ page }, use) => { await use(new LoginPage(page)); }`). In our test files, we import `test` from our custom fixture module and destructure `{ loginPage }` directly in the test function arguments. Playwright handles instantiation and lifecycle automatically, keeping tests clean and free of `new Class(page)` boilerplate.

Follow-Up Question: "Can custom fixtures have dependencies on other fixtures like storageState?"

Q4. What is the "God Object" anti-pattern in Page Object design, and how do you prevent it?

[Difficulty: Intermediate] Evaluation Focus: Class design, modularity, and single responsibility principle.

Model Answer: A "God Object" occurs when a developer puts all locators and methods for an entire application (or a huge multi-section dashboard) into a single massive Page Object class exceeding hundreds or thousands of lines. This violates the Single Responsibility Principle and causes merge conflicts. We prevent this by breaking pages into smaller, logical page objects and extracting shared elements (like navigation bars, footers, modal dialogs, and search bars) into independent Component Objects that can be composed into pages.

Follow-Up Question: "How does composition over inheritance apply to component objects?"

Q5. How do you structure a production-grade Playwright automation repository for a team of 10 QA engineers?

[Difficulty: Scenario-Based] Evaluation Focus: Repository layout, maintainability, scalability, and collaboration.

Model Answer: I structure the repository with clear separation of concerns: `playwright.config.js` at root; `pages/` for Page Object classes; `fixtures/` for `test.extend` fixtures and authenticated state setups; `test-data/` for JSON datasets and synthetic personas; `utils/` for generic helper libraries; and `tests/` organized by business domain feature folders (e.g. `tests/auth/`, `tests/checkout/`). We use linters, strict Git branching with pull request templates, and CI pipelines running headless execution with Trace Viewer artifacts on failure.

Follow-Up Question: "How do you prevent duplicate helper methods across different feature teams?"

1. What is the primary architectural purpose of the Page Object Model (POM)?

Key Concept: POM encapsulates locators and interactions within dedicated classes, so UI changes only require updating the Page Object rather than modifying dozens of test files.

2. How are element locators typically defined in a modern JavaScript Page Object?

Key Concept: Locators are defined in the constructor (e.g. this.submitButton = page.getByRole("button", ...)), keeping them centralized and ready for use in class methods.

3. Why should assertions like expect(locator).toHaveText() generally stay in the test spec rather than Page Object methods?

Key Concept: Keeping assertions in test files ensures that tests describe business requirements and allows Page Object methods to be reused across positive and negative scenarios.

4. Which Playwright feature allows you to inject Page Objects directly into tests without writing "new LoginPage(page)" repeatedly?

Key Concept: test.extend() enables dependency injection of Page Objects as fixtures, providing pre-instantiated pages directly in test function arguments.

5. What is a "God Object" in the context of Page Object design?

Key Concept: A God Object is an anti-pattern where a single class accumulates too many responsibilities, becoming difficult to navigate, maintain, and review.

6. Why is "Composition over Inheritance" preferred when designing reusable components like Navigation Bars or Footers?

Key Concept: Composition allows pages to include only the components they actually contain (e.g. this.header = new HeaderComponent(page)), avoiding brittle inheritance trees.

7. In a recommended Playwright project layout, where do Page Object classes belong?

Key Concept: Keeping Page Objects in a dedicated pages/ directory ensures clean separation from test specifications located in tests/.

8. What is wrong with writing custom wrapper methods like async function clickEl(locator) { await locator.click(); }?

Key Concept: Playwright locators already embody auto-waiting, visibility checks, and retry mechanics. Shallow wrapper functions obscure stack traces without adding any functionality.

9. If an application redesigns its checkout button from "Place Order" to "Pay Now", how many files must you update in a properly implemented POM framework?

Key Concept: The greatest advantage of POM is single-point maintenance: updating the locator in the Page Object automatically updates all test cases that interact with that button.

10. How should a Page Object method handle navigation to its corresponding page?

Key Concept: Providing an async goto() method encapsulates the page URL within the Page Object, allowing tests to navigate with a simple await myPage.goto() call.